In [ ]:
```python
import marimo as mo
import torch
import torch.nn as nn
import torch.nn.functional as F
import torchvision
import torchvision.transforms as transforms
import snntorch as snn
from snntorch import surrogate
from torch.utils.data import DataLoader
import matplotlib.pyplot as plt
```

In [ ]:
```python
# Параметры эксперимента
BATCH_SIZE = 128
EPOCHS = 3
NUM_STEPS = 20

INPUT_SIZE = 28 * 28
HIDDEN_SIZE = 128
OUTPUT_SIZE = 10

BETA = 0.9
device = torch.device(
    "cuda" if torch.cuda.is_available() else "cpu"
)

mo.md(
    f"""
    ## Полноценная LIF-SNN

    - Устройство: **{device}**
    - Входных нейронов: **{INPUT_SIZE}**
    - Скрытых нейронов: **{HIDDEN_SIZE}**
    - Выходных нейронов: **{OUTPUT_SIZE}**
    - Временных шагов: **{NUM_STEPS}**
    - Коэффициент утечки β: **{BETA}**
    """
)
```

Полноценная LIF-SNN

- Устройство: **cuda**
- Входных нейронов: **784**
- Скрытых нейронов: **128**
- Выходных нейронов: **10**
- Временных шагов: **20**
- Коэффициент утечки β: **0.9**

In [ ]:
```python
transform = transforms.Compose([
    transforms.ToTensor()
])
```

```
train_dataset = torchvision.datasets.MNIST(
    root="./data",
    train=True,
    download=True,
    transform=transform
)

test_dataset = torchvision.datasets.MNIST(
    root="./data",
    train=False,
    download=True,
    transform=transform
)

train_loader = DataLoader(
    train_dataset,
    batch_size=BATCH_SIZE,
    shuffle=True,
    num_workers=0
)

test_loader = DataLoader(
    test_dataset,
    batch_size=BATCH_SIZE,
    shuffle=False,
    num_workers=0
)

mo.md(
    f"""
    ## Данные

    - Обучающих изображений: {len(train_dataset)}
    - Тестовых изображений: {len(test_dataset)}
    - Размер изображения: 28 × 28
    """
)
```

Данные

- Обучающих изображений: 60000
- Тестовых изображений: 10000
- Размер изображения: 28 × 28

In [ ]:
```
class FullLIFNet(nn.Module):
    def __init__(self):
        super().__init__()

        # Синапсы: вход -> скрытый слой
        self.fc1 = nn.Linear(INPUT_SIZE, HIDDEN_SIZE)

        # Синапсы: скрытый слой -> выход
```

```
        self.fc2 = nn.Linear(HIDDEN_SIZE, OUTPUT_SIZE)

        # Все три слоя состоят из LIF-нейронов
        spike_grad = surrogate.fast_sigmoid(slope=25)

        self.lif_in = snn.Leaky(
            beta=BETA,
            spike_grad=spike_grad,
            reset_mechanism="subtract"
        )

        self.lif_hidden = snn.Leaky(
            beta=BETA,
            spike_grad=spike_grad,
            reset_mechanism="subtract"
        )

        self.lif_out = snn.Leaky(
            beta=BETA,
            spike_grad=spike_grad,
            reset_mechanism="subtract"
        )

    def forward(self, images):
        # images: [batch, 1, 28, 28]
        batch_size = images.shape[0]

        x = images.reshape(batch_size, -1)

        # Инициализируем состояния нейронов.
        # Состояния локальны для данного вызова forward.
        mem_in = torch.zeros_like(x)

        mem_hidden = torch.zeros(
            batch_size,
            HIDDEN_SIZE,
            device=x.device,
            dtype=x.dtype
        )

        mem_out = torch.zeros(
            batch_size,
            OUTPUT_SIZE,
            device=x.device,
            dtype=x.dtype
        )

        output_spikes = []

        for _ in range(NUM_STEPS):
            # Входные пиксели подаются как ток,
            # постоянный на протяжении временного окна.
            spk_in, mem_in = self.lif_in(x, mem_in)

            # На скрытый слой поступают спайки входного.
            cur_hidden = self.fc1(spk_in)
```

```
            spk_hidden, mem_hidden = self.lif_hidden(
                cur_hidden, mem_hidden
            )

            # Выходной слой получает спайки скрытого.
            cur_out = self.fc2(spk_hidden)

            spk_out, mem_out = self.lif_out(
                cur_out, mem_out
            )

            output_spikes.append(spk_out)

        # [время, batch, классы]
        return torch.stack(output_spikes, dim=0)

model = FullLIFNet()
```

In [ ]:

```
model.to(device)

optimizer = torch.optim.Adam(
    model.parameters(),
    lr=1e-3
)

criterion = nn.CrossEntropyLoss()

train_losses = []
train_accuracies = []
test_accuracies = []

def evaluate(model, loader):
    model.eval()

    correct = 0
    total = 0

    with torch.no_grad():
        for images, labels in loader:
            images = images.to(device)
            labels = labels.to(device)

            spikes = model(images)

            # Сколько раз спайкует каждый выходной нейрон
            spike_counts = spikes.sum(dim=0)

            predictions = spike_counts.argmax(dim=1)

            correct += (
                predictions == labels
            ).sum().item()

            total += labels.size(0)
```

```
    return 100.0 * correct / total

for epoch in range(EPOCHS):
    model.train()

    running_loss = 0.0
    correct = 0
    total = 0

    for images, labels in train_loader:
        images = images.to(device)
        labels = labels.to(device)

        optimizer.zero_grad(set_to_none=True)

        # Прямой проход по времени
        spikes = model(images)

        # [время, batch, классы] -> [batch, классы]
        spike_counts = spikes.sum(dim=0)

        loss = criterion(spike_counts, labels)

        # Обратное распространение через все временные шаги
        loss.backward()

        optimizer.step()

        running_loss += loss.item()

        predictions = spike_counts.detach().argmax(dim=1)

        correct += (
            predictions == labels
        ).sum().item()

        total += labels.size(0)

    mean_loss = running_loss / len(train_loader)
    train_acc = 100.0 * correct / total
    test_acc = evaluate(model, test_loader)

    train_losses.append(mean_loss)
    train_accuracies.append(train_acc)
    test_accuracies.append(test_acc)

    print(
        f"Epoch {epoch + 1}/{EPOCHS} | "
        f"loss={mean_loss:.4f} | "
        f"train_acc={train_acc:.2f}% | "
        f"test_acc={test_acc:.2f}%"
    )

mo.md("## Обучение завершено")
```

```
Epoch 1/3 | loss=0.3493 | train_acc=89.19% | test_acc=95.77%
Epoch 2/3 | loss=0.1238 | train_acc=96.19% | test_acc=96.62%
Epoch 3/3 | loss=0.0876 | train_acc=97.31% | test_acc=97.15%
```

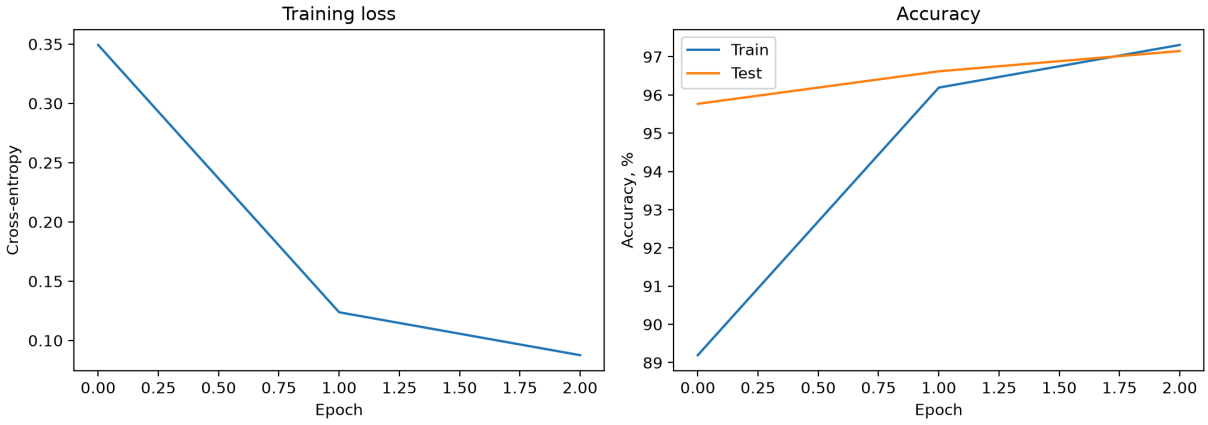
Обучение завершено

In [ ]:
```python
fig, axes = plt.subplots(1, 2, figsize=(11, 4))

axes[0].plot(train_losses)
axes[0].set_title("Training loss")
axes[0].set_xlabel("Epoch")
axes[0].set_ylabel("Cross-entropy")

axes[1].plot(train_accuracies, label="Train")
axes[1].plot(test_accuracies, label="Test")
axes[1].set_title("Accuracy")
axes[1].set_xlabel("Epoch")
axes[1].set_ylabel("Accuracy, %")
axes[1].legend()

fig.tight_layout()
plt.show()
```



In [ ]:
```python
def show_predictions():
    model.eval()

    count = 10
    sample_images = torch.stack([
        test_dataset[i][0] for i in range(count)
    ]).to(device)

    true_labels = torch.tensor([
        test_dataset[i][1] for i in range(count)
    ])

    with torch.no_grad():
        output_spikes = model(sample_images)
        spike_counts = output_spikes.sum(dim=0)
        predicted_labels = spike_counts.argmax(dim=1).cpu()
```

```
    fig, axes = plt.subplots(2, 5, figsize=(12, 5))

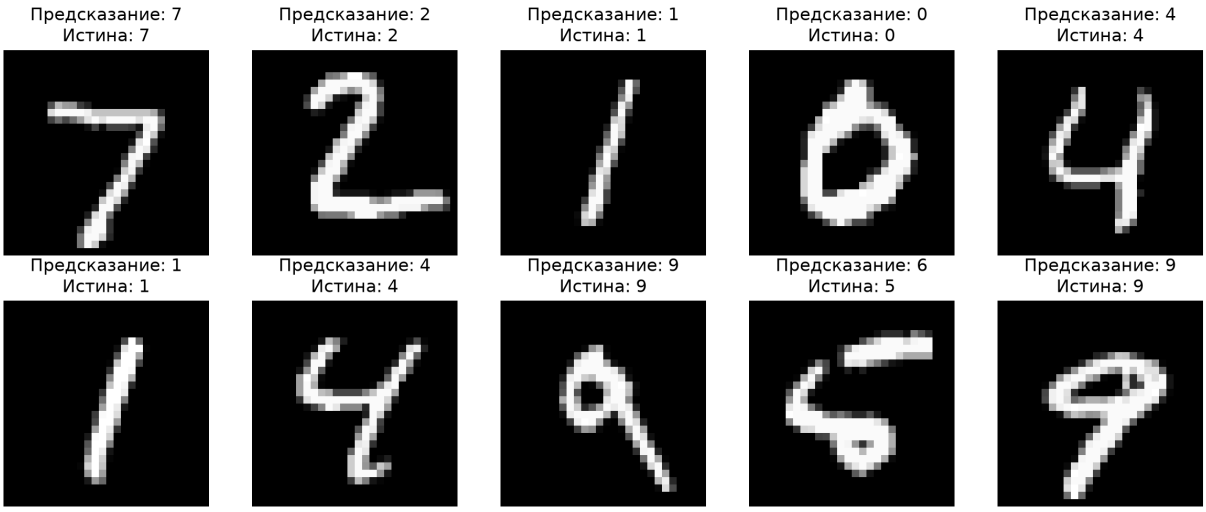
    for i, ax in enumerate(axes.flat):
        ax.imshow(
            sample_images[i, 0].cpu().numpy(),
            cmap="gray"
        )
        ax.set_title(
            f"Предсказание: {predicted_labels[i].item()}\n"
            f"Истина: {true_labels[i].item()}"
        )
        ax.axis("off")

    fig.tight_layout()
    plt.show()

show_predictions()

mo.md(
    """
    ## Как читать результат

    Выходной нейрон с наибольшим количеством спайков
    определяет предсказанную цифру.
    """
)
```



Как читать результат

Выходной нейрон с наибольшим количеством спайков определяет предсказанную цифру.